

Assignment Task 2

1. Extend the workload proposed in TASK 1, by adding at least 5 additional and structurally different requests, at your choice.

Q1. Given a club identifier and a competition identifier, determine the dates and stadium names of all games played by that club in that competition. **(Club)**

Q2. Given a country name and a player position, determine the names of all players from that country playing in that position, together with their current club names. **(Country)**

Q3. Given a stadium name, determine the identifiers, dates, and referees of all games played in that stadium. **(Stadium)**

Q4. Given a game-event type and a competition identifier, determine the players related to those events, together with the event minute and game identifier. **(Competition)**

Q5. Given a competition identifier, determine the competition type and the game belonging to that competition. **(Competition)**

Verification of the Constraints:

Constraints	Result	Relevant queries
At least five structurally different operations	Satisfied	Q1–Q5
Three aggregation entities	Satisfied	(Club, Country, Stadium, Competition)
At least three queries selecting on two entities	Satisfied	Q1, Q2, Q4
At least two paths with length ≥ 2	Satisfied	Q1 and Q4
Partition-key case (c)	Satisfied	Q4 and Q5

2. Use the aggregate-oriented design methodology (STEP 1-2-3) to design a set of logical aggregates for the given domain and the extended workload (operations proposed in TASK 1 + operations proposed in TASK 2). To this aim, revise the output of each step, presented for TASK 1, taking into account the new operations proposed for TASK 2.

Step 1 (Query Modeling):

Q1:

- E = Club
- LS = [Club(club_id)_! , Competition(competition_id)_H_H]
- LP = [Game(date)_G , Stadium(name)_Pi_H]

Q1 (Club, [Club(club_id)_!, Competition(competition_id)_ H_H], [Game(date)_G, Stadium(name)_Pi_H])

Q2:

- E = Country
- LS = [Country(name)_!, Player(position)_B]
- LP = [Player(name)_B , Club(name)_Cp_B]

Q2 (Country , [Country(name)_!, Player(position)_B], [Player(name)_B , Club(name)_Cp_B])

Q3:

- E = Stadium
- LS = [Stadium(name)_!]
- LP = [Game(game_id , date , referee)_Pi]

Q3 (Stadium, [Stadium(name)_!] , [Game(game_id, date , referee)_Pi])

Q4:

- E = Competition
- LS = [Competition(competition_id)_!, GameEvents(type)_G_H]
- LP = [GameEvents(minute)_G_H , Player(player_id)_RTo , Game(game_id)_H]

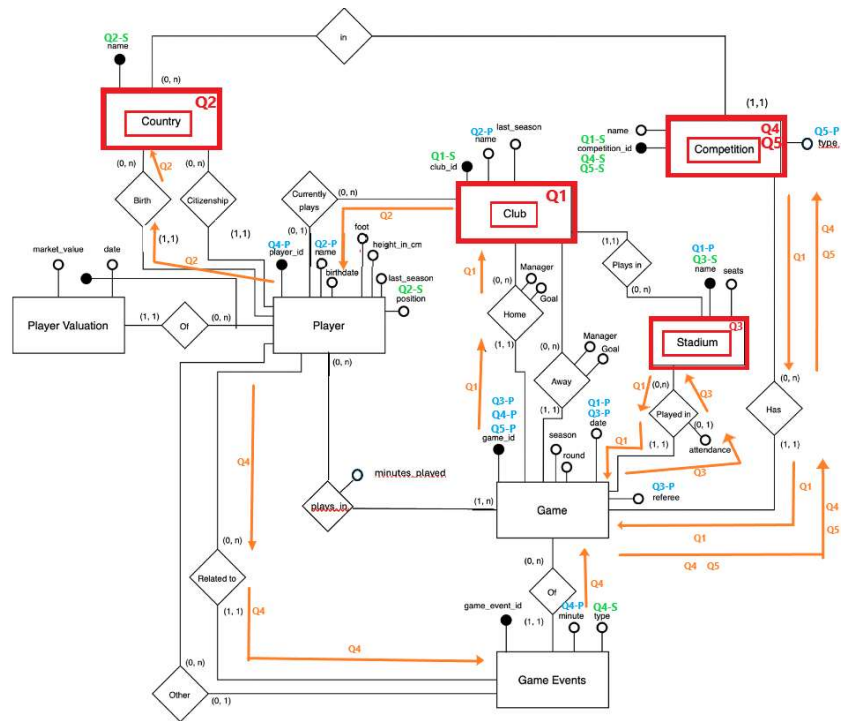
Q4 (Competition, [Competition(competition_id)_!, GameEvents(type)_G_H], [GameEvents(minute)_G_H, Player(player_id)_RTo , Game(game_id)_H])

Q5:

- E = Competition
- LS = [Competition(competition_id)_!]
- LP = [Competition(type)_!, Game(game_id)_H]

Q5 (Competition, [Competition(competition_id)_!], [Competition(type)_!, Game(game_id)_H])

Step 2 (Annotation of the ER Schema):



Step 3 (Translation into JSON Meta Notation):

Club Aggregate (Q1):

```
Club =
{
  club_id, name,
  games: [ { game_id, date, competition: { competition_id }, stadium: { name } } ]
}
```

Country Aggregate (Q2):

```
Country =
{
  name, players: [ { player_id, name, position, current_club: { club_id, name } } ]
}
```

Stadium Aggregate (Q3):

```
Stadium =
{
  name, games: [ { game_id, date, refereee } ]
}
```

Competition Aggregate (Q4 and Q5):

Competition =

```
{
  competition_id, type
  Games: [ { game_id, game_events: [ { type, minute, player: { player_id } } ] } ]
}
```

3. For each aggregate-oriented schema and each associated query, specify the query type (either batch or random access query).

Query	Aggregate	LS	Type
Q1	Club	club_id, competition_id	Random Access
Q2	Country	country_name, player_position	Random Access
Q3	Stadium	stadium_name	Random Access
Q4	Competition	competition_id, event_type	Random Access
Q5	Competition	competition_id	Random Access

All queries are Random Access Queries.

4. For each designed aggregate-oriented schema, specify a suitable partition key and a suitable super-identifier for the corresponding collection.

- **Club Aggregate (Q1):**

```
Club = { club_id p1, name, games: [ { game_id, date, competition: { competition_id }, stadium: { name } } ] }
```

Selection Attributes:

Q1 = {club_id, competition_id}

Case: (b) – Q1 is a random-access query and its simple selection attribute is club_id. Therefore, the intersection of the simple selection attributes is {club_id}, which can be used as the partition key. This follows the lecture definition of Case (b), where the random-access queries share the same set of simple selection attributes.

Partition Key: (club_id)

Super-Identifier: (club_id)

Since club_id is both the identifier of the Club aggregate and the partition key, no extension of the identifier is required.

- **Country Aggregate (Q2)**

```
Country = { name p1, players: [ { player_id, name, position, current_club: { club_id, name } } ] }
```

Q1 = {country_name, player_position}

Case: (b) – Q2 is a random-access query and its simple selection attribute is country_name. Therefore, the intersection of the simple selection attributes is {country_name}, which can be used as the partition key

Partition Key: (country_name)

Super-Identifier: (country_name)

player_position is not included in the partition key because it is nested inside the multivalued Players structure. Since country_name identifies the Country aggregate and is also the partition key, no extension is required.

- **Stadium Aggregate (Q3)**

Stadium = { name^{p1}, games: [{ game_id, date, refereee }] }

Selection Attributes:

Q4 = {stadium_name}

Case: (b) – Q3 is a random-access query and the set of simple selection attributes is {stadium_name}. Since Q3 is the only random-access query associated with the Stadium aggregate, the intersection is also {stadium_name}.

Partition Key: (stadium_name)

Super-Identifier: (stadium_name)

- **Competition Aggregate (Q4 and Q5):**

Competition = { competition_id^{p1}, type, Games: [{ game_id, game_events: [{type, minute, player: {player_id} }] }] }

Selection Attributes:

Q4 = {competition_id, event_type}

Q5 = {competition_id}

Case: (c) applies because the random-access queries have a non-empty intersection of simple selection attributes (competition_id), but Q4 contains an additional simple selection attribute (type). Therefore, the common attribute competition_id is selected as the partition key

Partition Key: date

Super-Identifier: (date, game_id)

5. For each designed aggregate-oriented schema and each associated query, specify if it can be executed efficiently on the collection.

Club Aggregate (Q1):

Q1: Not fully efficient. The condition on club_id identifies one node, but the query also contains a condition on competition_id, which is nested inside the multivalued Games structure and is not part of the partition key. Therefore, the relevant Club data may not be sequentially stored on that node.

- **Country Aggregate (Q2)**

Q2: Not fully efficient. The condition on `country_name` identifies one node, but the query also contains a condition on `player_position`, which is nested inside the multivalued `Players` structure and is not part of the partition key. Therefore, the relevant `Country` data may not be sequentially stored on that node.

- **Stadium Aggregate (Q3)**

Q3: Efficient. The query contains an equality condition on the complete partition key `stadium_name`. Therefore, one node is accessed and the required aggregates are sequentially stored.

- **Competition Aggregate (Q4 and Q5)**

Q4: Not fully efficient. The condition on `competition_id` identifies one node, but the query also contains a condition on `type`, which is not part of the partition key. Therefore, the relevant `Competition` aggregates may not be sequentially stored on that node.

Q5: Efficient. The query contains an equality condition on the complete partition key `competition_id`. Therefore, one node is accessed and the required aggregates are sequentially stored

6. For each designed aggregate-oriented schema, identify the indexes to be created to make the execution of each query associated with the collection efficient (if any).

- **Club Aggregate (Q1):**

A local ordered index must be created on the super-identifier: (`club_id`).
To improve Q1, a secondary local index should be created on: (`competition_id`)

- **Country Aggregate (Q2):**

A local ordered index must be created on the super-identifier: (`country_name`).
To improve Q2, a secondary local index should be created on: (`player_position`)

- **Stadium Aggregate (Q3):**

A local ordered index must be created on the super-identifier: (`stadium_name`).
No additional index is required for Q3 because it selects by the partition key and can already be executed efficiently.

- **Competition Aggregate (Q4 and Q5):**

A local ordered index must be created on the super-identifier: (`competition_id`). To improve Q4, a secondary local index should be created on: (`event_type`)

No additional index is required for Q5 because it selects by the partition key and can already be executed efficiently.

7. Design in MongoDB:

1. Translate the aggregate-oriented schema (including information about partition keys and indexes) into a MongoDB schema using the approaches/methodologies proposed in the course.
2. Specify each operation of the workload in the language supported by MongoDB

Part (a):

▪ Club (Q1)

Q1: Given a club identifier and a competition identifier, determine dates and stadium names of all games played by that club in that competition.

Q1 (Club, [Club(club_id)_!, Competition(competition_id)_H_H] , [Game(date)_G, Stadium(name)_Pi_H])

Club = { club_id, name, games: [{ game_id, date, competition: { competition_id }, stadium: { name } }] }

Schema:

```
db.createCollection("club")
db.club.createIndex( {"club_id": 1}, {unique: true} )
sh.shardCollection( "footballdataset.club", {"club_id": "hashed"} )
db.club.createIndex( {"games.competition.competition_id": 1}, {unique: false} )
```

▪ Country (Q2)

Q2: Given a country name and a player position, determine names of all players from that country playing in that position, together with current club names.

Q2 (Country, [Country(name)_!, Player(position)_B], [Player(name)_B, Club(name)_Cp_B])

Country = { name, players: [{ player_id, name, position, current_club: { club_id, name } }] }

Schema:

```
db.createCollection("country")
db.country.createIndex( {"name": 1}, {unique: true} )
sh.shardCollection( "footballdataset.country", {"name": "hashed"} )
db.country.createIndex( {"players.position": 1}, {unique: false} )
```

▪ Stadium (Q3)

Q3: Given a stadium name, determine identifiers, dates, and referees of all games played in that stadium.

Q3 (Stadium, [Stadium(name)_!], [Game(game_id, date, referee)_Pi])

Stadium = { name, seats, games: [{ game_id, date, referee }] }

Schema:

```
db.createCollection("stadium")
db.stadium.createIndex( {"name": 1}, {unique: true} )
sh.shardCollection( "footballdataset.stadium", {"name": "hashed"} )
```

- **Competition (Q4 and Q5)**

Q4: Given a game-event type and a competition identifier, determine the players related to those events, together with the event minute and game identifier.

Q4 (Competition, [Competition(competition_id)_!, GameEvents(type)_G_H], [GameEvents(minute)_G_H, Player(player_id)_RTo, Game(game_id)_H])

Q5: Given a competition identifier, determine the competition type and the game belonging to that competition.

Q5 (Competition, [Competition(competition_id)_!], [Competition(type)_!, Game(game_id)_H])

Competition = { competition_id, type, games: [{ game_id, game_events: [{ type, minute, player: { player_id } }] }] }

Schema:

```
db.createCollection("competition")
db.competition.createIndex( {"competition_id": 1}, {unique: true} )
sh.shardCollection( "footballdataset.competition", {"competition_id": "hashed"} )
db.competition.createIndex( {"games.game_events.type": 1}, {unique: false} )
```

Part (b):

- **Q1:** Given a club identifier and a competition identifier, determine dates and stadium names of all games played by that club in that competition.

```
db.club.aggregate ( [
  {"$match": { "club_id": 1 }},
  {"$unwind": "$games"},
  {"$match": { "games.competition.competition_id": 10 }},
  {"$project": { "_id": 0, "date": "$games.date", "stadium_name": "$games.stadium.name" }}
] )
```

- **Q2:** Given a country name and a player position, determine names of all players from that country playing in that position, together with current club names.

```
db.country.aggregate([
  {"$match": { "name": "Italy" }},
  {"$unwind": "$players"},
  {"$match": { "players.position": "Goalkeeper" }},
  {"$project": { "name": "$players.name", "club": "$players.club" }}
])
```

```
{ "$match": { "players.position": "Midfield" } },
{ "$project": { "_id": 0, "player_name": "$players.name", "current_club_name":
"$players.current_club.name" } }
})
```

- **Q3:** Given a stadium name, determine identifiers, dates, and referees of all games played in that stadium.

```
db.stadium.find( { "name": "Stadium A"}, { "_id": 0, "games.game_id": 1, "games.date": 1,
"games.referee": 1 } )
```

- **Q4:** Given a game-event type and a competition identifier, determine the players related to those events, together with the event minute and game identifier.

```
db.competition.aggregate([
{ "$match": { "competition_id": 10 } },
{ "$unwind": "$games"},
{ "$unwind": "$games.game_events"},
{ "$match": { "games.game_events.type": "Goal"}},
{ "$project": { "_id": 0, "player_id": "$games.game_events.player.player_id",
"minute": "$games.game_events.minute", "game_id": "$games.game_id" } }
])
```

- **Q5:** Given a competition identifier, determine the competition type and the game belonging to that competition.

```
db.competition.find( { "competition_id": 10}, { "_id": 0, "type": 1, "games.game_id": 1 } )
```

8. Design in Cassandra

1. Translate the aggregate-oriented schema (including information about partition keys and indexes) into a Cassandra schema, using the approaches/methodologies proposed in the course.
2. Specify in CQL each operation of the workload.

Part (a):

- **Club (Q1)**

Q1: Given a club identifier and a competition identifier, determine dates and stadium names of all games played by that club in that competition.

Q1 (Club, [Club(club_id)_!, Competition(competition_id)_H_H] , [Game(date)_G, Stadium(name)_Pi_H])

Club = { club_id, name, games: [{ game_id, date, competition: { competition_id }, stadium: { name } }] }

Schema:

```
CREATE TYPE competition_type ( competition_id int );
```

```
CREATE TYPE stadium_type ( name text );
```

```
CREATE TYPE club_game_type ( game_id int, date date,  
competition frozen<competition_type>, stadium frozen<stadium_type> );
```

```
CREATE TABLE club (  
club_id int,  
name text,  
games list<frozen<club_game_type>>,  
PRIMARY KEY (club_id) );
```

- **Country (Q2)**

Q2: Given a country name and a player position, determine names of all players from that country playing in that position, together with current club names.

Q2 (Country, [Country(name)_!], Player(position)_B], [Player(name)_B, Club(name)_Cp_B])

Country = { name, players: [{ player_id, name, position, current_club: { club_id, name } }] }

Schema:

```
CREATE TYPE current_club_type ( club_id int, name text );
```

```
CREATE TYPE country_player_type ( player_id int, name text, position text,  
current_club frozen<current_club_type> );
```

```
CREATE TABLE country (  
name text,  
players list<frozen<country_player_type>>,  
PRIMARY KEY (name) );
```

- **Stadium (Q3)**

Q3: Given a stadium name, determine identifiers, dates, and referees of all games played in that stadium.

Q3 (Stadium, [Stadium(name)_!], [Game(game_id, date, referee)_Pi])

Stadium = { name, seats, games: [{ game_id, date, referee }] }

Schema:

```
CREATE TYPE stadium_game_type ( game_id int, date date, referee text );
```

```
CREATE TABLE stadium (  
name text,  
seats int,  
games list<frozen<stadium_game_type>>,  
PRIMARY KEY (name) );
```

▪ **Competition (Q4 and Q5)**

Q4: Given a game-event type and a competition identifier, determine the players related to those events, together with the event minute and game identifier.

Q4 (Competition, [Competition(competition_id)_!], GameEvents(type)_G_H], [GameEvents(minute)_G_H, Player(player_id)_RTo, Game(game_id)_H])

Q5: Given a competition identifier, determine the competition type and the game belonging to that competition.

Q5 (Competition, [Competition(competition_id)_!], [Competition(type)_!, Game(game_id)_H])

Competition = { competition_id, type, games: [{ game_id, game_events: [{ type, minute, player: { player_id } }] }] }

Schema:

```
CREATE TYPE event_player_type ( player_id int );
```

```
CREATE TYPE game_event_type ( type text, minute int, player frozen<event_player_type> );
```

```
CREATE TYPE competition_game_type ( game_id int, game_events list<frozen<game_event_type>> );
```

```
CREATE TABLE competition (  
competition_id int,  
type text,  
games list<frozen<competition_game_type>>,  
PRIMARY KEY (competition_id) );
```

Part (b):

- **Q1:** Given a club identifier and a competition identifier, determine dates and stadium names of all games played by that club in that competition.

```
SELECT games FROM club WHERE club_id = 1;
```

- **Q2:** Given a country name and a player position, determine names of all players from that country playing in that position, together with current club names.

SELECT players FROM country WHERE name = 'Italy';

- **Q3:** Given a stadium name, determine identifiers, dates, and referees of all games played in that stadium.

SELECT games FROM stadium WHERE name = 'Stadium A';

- **Q4:** Given a game-event type and a competition identifier, determine the players related to those events, together with the event minute and game identifier.

SELECT games FROM competition WHERE competition_id = 10;

- **Q5:** Given a competition identifier, determine the competition type and the game belonging to that competition.

SELECT type, games FROM competition WHERE competition_id = 10;

9. Design in Neo4J

1. Design a schema for Neo4j, using the approaches/methodologies proposed in the course.
2. Specify in Neo4j each operation of the workload.

Part (a):

- **CONSTRAINTS**

CREATE CONSTRAINT FOR (c:Club)
 REQUIRE c.club_id IS UNIQUE;

CREATE CONSTRAINT FOR (p:Player)
 REQUIRE p.player_id IS UNIQUE;

CREATE CONSTRAINT FOR (g:Game)
 REQUIRE g.game_id IS UNIQUE;

CREATE CONSTRAINT FOR (s:Stadium)
 REQUIRE s.name IS UNIQUE;

CREATE CONSTRAINT FOR (comp:Competition)
 REQUIRE comp.competition_id IS UNIQUE;

CREATE CONSTRAINT FOR (con:Country)
 REQUIRE con.name IS UNIQUE;

```
CREATE CONSTRAINT FOR (e:GameEvent)
  REQUIRE e.game_event_id IS UNIQUE;
```

- **Nodes**

```
CREATE (:Club { club_id: 1, name: 'Genoa CFC' });
CREATE (:Player { player_id: 1, name: 'Player A', position: 'Midfield' });
CREATE (:Country { name: 'Italy' });
CREATE (:Stadium { name: 'Luigi Ferraris', seats: 33205 });
CREATE (:Game { game_id: 1, date: date('2026-01-12'), referee: 'Referee A' });
CREATE (:Competition { competition_id: 1, type: 'Domestic League' });
CREATE (:GameEvent { game_event_id: 1, minute: 35, type: 'Goal' });
```

- **Relationships**

- **Club plays in Game**

```
MATCH (c:Club {club_id: 1}), (g:Game {game_id: 1})
CREATE (c)-[:PLAYS_IN]->(g);
```

- **Game belongs to Competition**

```
MATCH (g:Game {game_id: 1}), (comp:Competition {competition_id: 1})
CREATE (g)-[:BELONGS_TO]->(comp);
```

- **Game is played at Stadium**

```
MATCH (g:Game {game_id: 1}), (s:Stadium {name: 'Luigi Ferraris'})
CREATE (g)-[:PLAYED_AT]->(s);
```

- **Player belongs to Country**

```
MATCH (p:Player {player_id: 1}), (con:Country {name: 'Italy'})
CREATE (p)-[:BORN_IN]->(con);
```

- **Player currently plays for Club**

```
MATCH (p:Player {player_id: 1}), (c:Club {club_id: 1})
CREATE (p)-[:PLAYS_FOR]->(c);
```

- **Game has GameEvent**

```
MATCH (g:Game {game_id: 1}), (e:GameEvent {game_event_id: 1})
CREATE (e)-[:IN_GAME]->(g);
```

- **GameEvent is related to Player**

```
MATCH (e:GameEvent {game_event_id: 1}), (p:Player {player_id: 1})
CREATE (e)-[:BY_PLAYER]->(p);
```

Part (b):

- **Q1:** Given a club identifier and a competition identifier, determine dates and stadium names of all games played by that club in that competition.

```
MATCH (c:Club {club_id: 1}) - [:PLAYS_IN] -> (g:Game) - [:BELONGS_TO] -> (:Competition
{competition_id: 1}),
(g) - [:PLAYED_AT] -> (s:Stadium)
RETURN g.date, s.name AS stadium_name;
```

- **Q2:** Given a country name and a player position, determine names of all players from that country playing in that position, together with current club names.

```
MATCH (p:Player {position: 'Midfield'}) - [:BORN_IN] -> (:Country {name: 'Italy'}),
(p) - [:PLAYS_FOR] -> (c:Club)
RETURN p.name AS player_name, c.name AS current_club_name;
```

- **Q3:** Given a stadium name, determine identifiers, dates, and referees of all games played in that stadium.

```
MATCH (g:Game) - [:PLAYED_AT] -> (s:Stadium {name: 'Luigi Ferraris'})
RETURN g.game_id, g.date, g.referee;
```

- **Q4:** Given a game-event type and a competition identifier, determine the players related to those events, together with the event minute and game identifier.

```
MATCH (e:GameEvent {type: 'Goal'}) - [:IN_GAME] -> (g:Game) - [:BELONGS_TO] -> (:Competition
{competition_id: 1}),
(e)-[:BY_PLAYER]->(p:Player)
RETURN p.player_id, e.minute, g.game_id;
```

- **Q5:** Given a competition identifier, determine the competition type and the game belonging to that competition.

```
MATCH (g:Game) - [:BELONGS_TO] -> (comp:Competition {competition_id: 1})
RETURN comp.type, g.game_id;
```

10. Discuss which, among the three systems, is the most suitable to be used as back-end for your application, taking into account the identified workload (step 4). Let S be the selected system.

In our workload, all queries are random-access queries based on selection attributes such as club_id, competition_id, country name, stadium name, player position, and event type.

Neo4j is suitable for highly connected data and graph traversal, but our workload mainly retrieves data already grouped inside aggregates. Cassandra is less suitable because some selection attributes are stored inside nested UDTs, requiring additional application-level filtering in some queries. MongoDB fits our workload better because our aggregates naturally map to nested JSON documents, and MongoDB supports flexible queries and indexes on nested fields.

Therefore, considering the workload and aggregate design, MongoDB is the most suitable back-end system.

Selected system S = MongoDB.

11. Create the logical schema in system S (for MongoDB and Cassandra, use the cluster).

12. Create an instance of your schema in the selected system, according to the logical schema just created, starting from data available here. Notice that:

- 1. The dataset should have a reasonable size (few Mb).**
- 2. The dataset might need to be transformed in order to be used by your application. For dataset transformation, you can rely on either data transformation tools, such as Tableaux Prep (www.tableau.com), Apache Superset (superset.apache.org) Trifacta (www.trifacta.com), other ETL tools such as Talend (www.talend.com), or scripts in your favorite language.**
- 3. For importing datasets in the chosen system, you should refer to the available documentation for the system you have selected (e.g. <https://www.datastax.com/dev/blog/simple-data-importing-and-exporting-with-cassandra> for Cassandra and <https://neo4j.com/developer/guide-importing-data-and-etl> for Neo4J).**

13. Implement the workload in system S.

DISCLOSURE: I used a Large Language Model (LLM) to improve the wording of the natural language queries and to help generate and understand the partition key and super-identifier, as applying the partition key selection cases to the queries was sometimes confusing. I also used an LLM to assist with designing the system in MongoDB, Cassandra, and Neo4j, particularly with the syntax and complex queries. The final submission reflects my own understanding after reviewing the lecture material.